



CPPRaft系列-常见问题及解答-07

来自：代码随想录

思无邪

2024年02月05日 22:06

第七篇-问题解答

[toc]

没想到吧，我又回来了。

目前项目中还有很多地方可以优化，欢迎大家参与吼吼吼。

地址在：

<https://github.com/youngyangyang04/KVstorageBaseRaft-cpp>

在前面的系列文章中，我对这个项目提出了很多问题，但是发现没有解答，现在是时候回收伏笔，对这些问题做一个解答了。

notice：分布式知识需要严谨的求证，本人学艺不精只能尽可能的保证下面的回答没有问题。如果发现有疏漏的地方欢迎指出，共同进步，不胜感谢。

Raft

Raft算法的基本原理：

回答要点：解释Raft算法的基本工作原理，包括领导者选举、日志复制和安全性保障。

示例回答：

Raft算法是一种分布式算法，旨在解决分布式系统中的一致性问题，相对于Paxos算法而言更易于理解和实现。

Raft算法将系统中的所有节点分为三类角色：领导者（leader）、跟随者（follower）和候选人（candidate）。其选举机制确保系统中的一个节点被选为领导者（leader），领导者负责处理客户端的请求，并将更新复制到其他节点。

Raft算法的基本原理包括以下几个关键步骤：

1. 领导者选举（Leader Election）：在系统启动时或者当前领导者失效时，节点会发起选举过程。节点会在一个随机的超时时间内等待收到来自其他节点的心跳消息。如果在超时时间内没有收到心跳消息，节点就会成为候选人并发起选举。候选人向其他节点发送投票请求，并在得到大多数节点的投票后成为新的领导者。
2. 日志复制（Log Replication）：一旦领导者选举完成，新的领导者就会接收客户端的请求，并将更新的日志条目复制到其他节点。当大多数节点都成功复制了这些日志条目时，更新被认为是提交的，并且可以应用到节点的状态机中。
3. 安全性（Safety）：Raft算法通过确保在选举中只有一个领导者（单一领导者）、大多数节点的一致性以及只有领导者可以处理客户端请求等方式保证分布式系统的安全性。

通过以上机制，Raft算法确保了分布式系统中的一致性、可用性和分区容错性。

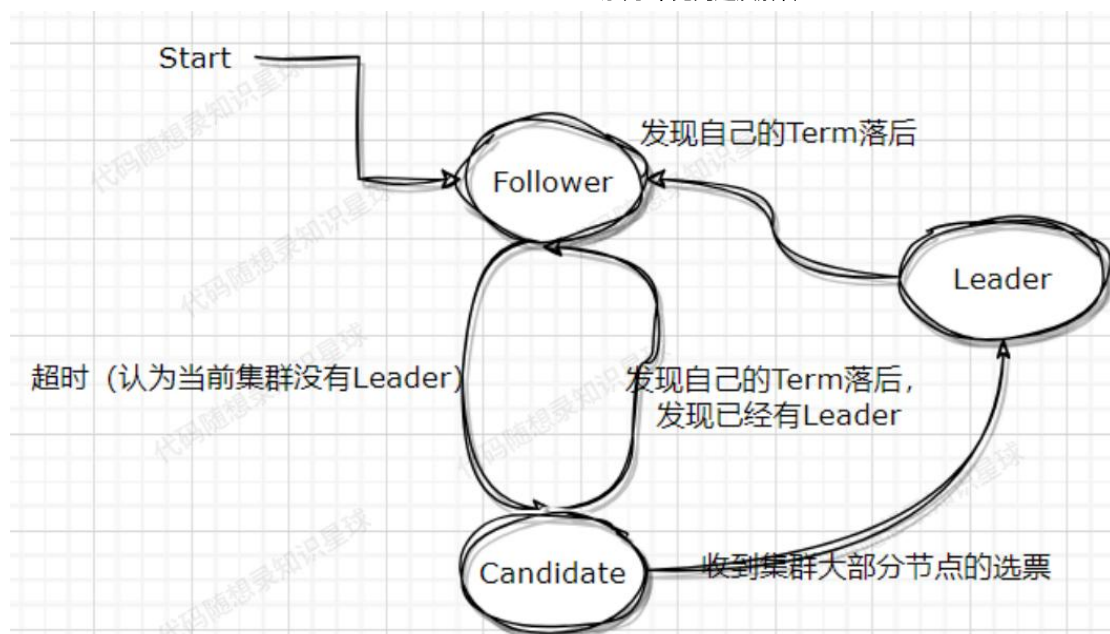
注意：如果这么回答如果面试官懂一些分布式算法的话，那么后续可能会提问Raft与其他分布式算法的关系。

领导者选举：

如何进行Raft中的领导者选举？

回答要点：这里最好结合前面几个章节的流程图，结合自己理解回答。

示例回答：



Raft中的领导者选举过程如下：

1. 候选人状态 (Candidate State) :

- 节点在没有检测到领导者的情况下成为候选人，并将自己的任期编号 (term) 增加1。
- 候选人向其他节点发送投票请求，并请求其他节点投票支持自己成为新的领导者。
- 如果候选人在规定时间内收到了大多数节点的选票支持（即获得了大多数节点的投票），则成为新的领导者。

2. 选举过程 (Election Process) :

- 在发起选举后，候选人会等待一定的随机时间（选举超时时间）来收集其他节点的投票。
- 如果在这个超时时间内没有收到大多数节点的选票，候选人将会重新开始一个新的选举周期，**增加自己的任期编号，并再次发起选举**。

3. 投票过程 (Voting Process) :

- 其他节点收到来自候选人的投票请求后，会检查自己的当前任期编号。如果候选人的任期编号比自己的大，则投票支持候选人，并更新自己的任期编号为候选人的任期编号。
- 如果其他节点已经投票给了另一个候选人，或者已经投票给了当前领导者，它将拒绝投票。

4. 领导者选举完成 (Leader Election Complete) :

- 如果候选人收到了大多数节点的投票支持，它就会成为新的领导者。
- 新的领导者会开始发送心跳消息以维持其领导地位，并开始进行日志复制操作。

在什么情况下会触发领导者选举？

回答要点：一个节点只要长时间没有收到符合条件的leader发送的心跳就会认为leader掉线，就会发起选举。

示例回答：

在Raft算法中，领导者选举会在以下情况下触发：

1. 当系统启动时，所有节点都处于初始状态，没有领导者。
2. 当领导者节点因网络分区、宕机或其他原因失效时，导致系统中没有活跃的领导者。
3. 当节点故障恢复或者被网络分区时，它可能会检测到当前没有领导者，因此会成为候选人并发起选举。

日志复制：

Raft是如何通过日志复制来保证数据一致性的？

回答要点：主要是两个机制（特点）：

1. Leader Append Entries: 领导者追加日志条目，即**只有leader可以接受外部请求并将请求打包成日志**，并向follower同步自己的日志，这样保证提交过的日志不会被覆盖掉。
2. commit机制，领导者发现**大多数节点**都已经成功复制了某个日志条目后，该日志条目被视为已经提交，从而保证了数据的一致性。

示例回答：

Raft算法通过日志复制来确保数据一致性。在Raft中，每个节点都维护一个日志 (log) 来记录状态机中的操作指令。领导者负责接收客户端的写请求，将操作指令追加到自己的日志中，并将这些操作指令发送给其他节点，要求它们复制这些日志条目。

以下是Raft通过日志复制来保证数据一致性的基本流程：

1. Leader Append Entries (领导者追加日志条目) :

- 领导者接收到客户端的写请求后，将这些操作指令追加到自己的日志中。

- 领导者将这些操作指令组织成一个日志条目 (log entry)，并向其他节点发送一个追加日志条目的请求 (Append Entries RPC)。

2. Follower Log Replication (跟随者日志复制)：

- 跟随者节点接收到领导者发送的追加日志条目的请求后，会按照领导者的日志条目顺序将这些日志条目追加到自己的日志中。
- 如果跟随者节点成功复制了这些日志条目，则向领导者发送成功响应 (Response)；如果由于某种原因 (例如网络故障) 导致复制失败，则向领导者发送失败响应。

3. Commit (提交)：

- 当领导者发现大多数节点都已经成功复制了某个日志条目后，该日志条目被视为已经提交。
- 领导者将提交的日志条目应用到自己的状态机中，以执行相应的操作指令。

安全性保障：

Raft是如何确保安全性的？讨论一致性、可用性和分区容错性之间的权衡。

回答要点： 这里主要是想考察分布式CAP理论的一个关键：CAP中如果发生故障，只能CP和AP二选一，无法满足CAP的三角，而Raft选择的是CP，即满足一致性。

示例回答：

在权衡一致性、可用性和分区容错性时，Raft算法倾向于优先保证一致性和分区容错性。它通过保证大多数节点的确认和限制领导者选举条件来确保一致性，通过选举机制和日志复制来保证分区容错性。同时，Raft也兼顾了系统的可用性，确保在领导者失效后能够快速进行新的领导者选举，并继续提供服务。

选举超时：

什么是选举超时？它的作用是什么？

回答要点： follower和candidate都会有选举超时的机制。

在follower时：选举超时的意义是发起选举，变成candidate；

在candidate时：candidate会选举超时，如果选举成功就会变成leader；如果选举失败就会变成candidate（选举超时）或者follower（发现合适的leader）。那么选举超时的作用就很明显了，防止无止境的等待导致所有人都成不了leader。

拓展：想一想为什么选举超时时间要每次随机设置而不设置成一个固定的值？？？

示例回答：

选举超时的作用包括：

1. **触发领导者选举：** 选举超时用于在当前没有活跃领导者或者领导者失效时触发新的领导者选举。当节点在选举超时时间内没有收到来自当前领导者的心跳消息时，会成为候选人并发起选举过程。
2. **防止脑裂 (Split-Brain)：** 选举超时帮助避免了系统中出现多个领导者的情况，从而避免了脑裂问题的发生。如果系统中的节点在选举超时时间内没有收到来自当前领导者的心跳消息，它们会同时成为候选人并发起选举，但只有一个候选人最终会获得大多数节点的选票，成为新的领导者。
3. **确保领导者切换的及时性：** 选举超时可以确保在领导者失效后，系统能够及时地启动新的领导者选举过程，从而减少服务中断的时间，提高系统的可用性。

选举超时的时间是如何设置的？

****回答要点：** **回答要点在上个问题的拓展里面，大家可以先想想。答案是：一个一定范围内的随机值，其要根据心跳时间，rpc延迟，数据操作延迟综合考虑。

范围：一般来说选举超时时间要大于一次完整心跳的日志同步处理时间。

为何随机：选举超时的目的是防止无止境的等待导致所有人都成不了leader，如果超时时间又一样，那么大家又一起选举，又会不断循环，那么一个随机值可以让某些节点早点重新发起选举，防止大家一起选举导致死循环。

示例回答：

选举超时时间的设置通常包括以下考虑因素：

1. **网络延迟和稳定性：** 选举超时时间需要足够长以允许节点在正常情况下能够收到来自领导者的心跳消息。考虑到网络的延迟和不稳定性，超时时间应该设置得足够长，以避免因网络延迟而误判领导者失效。
2. **系统负载和响应速度：** 选举超时时间也应考虑系统的负载情况和响应速度。如果系统负载较重或者节点的处理速度较慢，可能需要将选举超时时间设置得稍长一些，以允许节点有足够的时间处理收到的消息。
3. **避免脑裂问题：** 为了避免系统中出现多个领导者导致的脑裂问题，选举超时时间应该设置得足够随机化，以确保不同节点不会在同一时间内触发选举。

日志条目的提交：

Raft中的日志条目是如何提交的？

回答要点： 要半数以上的节点（包括leader）接收了这个日志，那么才能提交（commit），后续才能apply到状态机。

示例回答：

1. Leader接收客户端请求：

- 当客户端向Raft系统提交请求时，请求会首先发送到Raft集群中的Leader节点。

2. Leader将请求转换为日志条目：

- Leader将接收到的客户端请求转换为一条日志条目，并附加到其本地日志中。
3. **Leader广播日志条目：**
 - Leader向其它节点发送包含新日志条目的心跳RPC请求（AppendEntries RPC）。
 4. **Follower节点接收并附加日志条目：**
 - Follower节点接收到Leader的附加日志请求后，将新的日志条目附加到其本地日志中。
 5. **Follower节点响应Leader：**
 - Follower节点在成功附加日志后，向Leader发送成功的响应。
 6. **Leader确认提交：**
 - 当Leader收到大多数节点的附加成功响应时，将日志条目视为已提交。
 7. **Leader提交到状态机：**
 - Leader将已提交的日志条目应用到其状态机中，以执行相应的操作。
 8. **Leader通知Followers提交：**
 - Leader会通知其它节点已提交的日志索引，以便它们也可以将相应的日志条目提交到其状态机中。
 9. **Follower提交到状态机：**
 - Follower节点收到Leader的提交通知后，将对应的已提交日志条目应用到其状态机中。

什么条件下才能够提交一个日志条目？

回答要点： 一个很容易疏忽的是必须要本term有新的日志提交才能继续提交日志，这个在前面文章中也提醒过。

示例回答： 略。

拓扑变更：

关于Raft节点变更属于比较进阶的知识，面试中没遇到过，建议大家可以了解一下

此外，这块我掌握的也不是很好，可能有错误的地方，欢迎大家指正。

Raft如何处理集群拓扑的变更？

回答要点：

示例回答：

1. **生成配置变更请求：** 当需要改变集群的拓扑结构时，例如添加或移除节点，Leader节点会收到来自客户端的配置变更请求。
2. **将配置变更请求转化为配置变更日志条目：** Leader节点将配置变更请求转换为一条特殊的日志条目，即配置变更日志条目。该日志条目包含新的集群配置信息。
3. **向集群中的节点分发配置变更日志条目：** Leader节点通过Raft的日志复制机制将配置变更日志条目发送给所有的Follower节点。
4. **Follower节点接收配置变更日志条目：** Follower节点收到配置变更日志条目后，将其附加到本地日志中。
5. **提交配置变更日志条目：** 当配置变更日志条目被大多数节点确认并附加到本地日志后，Leader节点将其提交到状态机，实际应用这个配置变更。
6. **应用配置变更：** Leader节点和所有的Follower节点根据新的配置信息更新其内部状态，以反映集群拓扑结构的变更。这可能涉及到更新节点的角色（Leader、Follower、Candidate）、维护新的节点列表等。

实际应用：

Raft算法在实际场景中的应用有哪些？

回答要点： 列举一些常见的使用Raft算法作为底层的即可。

示例回答：

1. 一些常见的配置中心，为了保证可用会采用Raft，比如zookeeper的底层实现了基于Raft修改的算法，ETCD等。
2. 一些分布式数据库，比如TIKV

Raft与其他分布式的比较：

与Paxos算法相比，Raft有哪些优势和不同之处？

回答要点： Raft相对于Paxos算法来说，更易理解、实现和维护，具有更直观的Leader机制和选举过程。（这个需要大家多了解一下其他分布式算法的设计思想了。）

示例回答：

1. **Leader机制：** **Raft引入了Leader节点，而Paxos中没有Leader节点的概念。**Leader节点负责协调和领导整个一致性过程，而Follower节点只需按照Leader的指示执行操作。
2. **日志复制：** 在Raft中，所有的日志条目都通过Leader节点进行复制和提交，而Paxos中的日志复制是通过多个角色相互协作完成的。
3. **角色切换：** Raft中Leader节点失效后，集群可以快速选举新的Leader节点，而Paxos中角色的切换较为复杂，需要进行更多的投票和协调。
4. **更强的可读性：** Raft算法更加直观和易于理解，它的设计目标之一就是提供更好的可读性和可理解性，相比之下，Paxos算法相对更加抽象和复杂。

常见问题与挑战：

Raft算法在分布式系统中有哪些常见的问题和挑战？

回答要点： leader的瓶颈（使用多读来解决），节点故障等等

示例回答：

1. **Leader瓶颈：** Raft算法中的Leader节点负责所有的客户端请求的处理和日志复制，这可能会成为系统的瓶颈。如果Leader节点负载过重或者发生故障，会导致整个系统的性能下降。
2. **网络分区：** Raft算法需要保证在网络分区情况下的一致性，这可能会导致在网络分区恢复后需要进行数据的合并和冲突解决，增加了一致性维护的复杂性。
3. **节点故障处理：** 当节点发生故障时，Raft需要进行Leader的重新选举，这可能导致一段时间内系统的不可用性和性能下降，尤其是在节点频繁发生故障时。
4. **日志复制延迟：** Raft算法要求日志复制必须在大多数节点上完成后才能提交，这可能导致日志复制的延迟，影响系统的实时性能。
5. **节点动态变更：** Raft算法在节点动态加入或退出时需要进行配置变更，这可能会导致系统的不稳定和数据的不一致，需要谨慎处理。
6. **一致性保证：** 虽然Raft算法保证了强一致性，但在一些特殊情况下（如网络分区、节点故障等），可能会导致一致性级别的降低或者一致性协议的不满足，需要额外的机制来解决。
7. **性能调优：** 在实际应用中，Raft算法需要根据具体的场景进行性能调优，包括调整心跳超时时间、选举超时时间、日志复制的策略等参数，以满足系统的性能需求。

如何处理网络分区的情况？

回答要点： 这个要结合多种情况分析，比如leader宕机、非leader宕机；少数节点分区、多数节点分区。然后这几种情况还可以相互组合，这个的话就要分类讨论了，面试估计是说不完的。

示例回答：

分情况讨论，略。

容错性：

Raft算法如何处理节点故障？

回答要点：

和网络分区是一样的，可以看成是网络分区的一种特殊情况，即一个节点自己是一个分区。

此外再加上故障恢复后有哪些数据（日志，投票，term等）是需要持久化的，哪些是不需要的（commitIndex，applyIndex等）。

示例回答：

略。

在集群中的多个节点同时故障时，系统会有什么表现？

****回答要点：** **考虑故障节点的数量，抓住“半数”这个概念。其他同上上面的“分区问题”和“故障问题”。

示例回答： 略。

RPC

你的RPC如何设计的？

回答要点： 这里最好的回答是回答出现有的rpc框架的一些优秀的设计，因为我的rpc实现只是raft-kv的一个配件，所以还是 同步阻塞 的，这里推荐大家看看异步rpc如何实现，也欢迎来改进项目中的RPC实现参与开源：[项目地址](#)
列出可以考虑的优化点：异步；rpc长连接短连接的思考；**负载均衡**；服务治理、服务发现。

示例回答：

略

负载均衡有没有做？用的什么算法如何考虑的？

回答要点： 1.常见的负载均衡算法及其对比；2.第四层和第七层不同层的负载均衡；3.瘦客户端和胖客户端的不同方式的负载均衡。

示例回答：

最开始实现rpc模块的时候实现过负载均衡算法，当然后面用于raft通信，因为raft通信是所有leader与所有节点都要建立连接，所以后面就没有再用负载均衡了，将这个功能关闭了。

当时实现的负载均衡的算法有：

1. **轮询算法（Round Robin）：**

- 轮询算法是最简单的负载均衡算法之一，它按照请求的顺序依次将每个请求分配到不同的服务器上。当有新的请求到来时，负载均衡器会依次将请求发送到不同的服务器，直到所有的服务器都被轮询过一遍，然后再从头开始。

2. **最小连接数算法（Least Connections）：**

- 最小连接数算法会将新的请求分配到当前连接数最少的服务器上，以确保各服务器的负载尽可能均衡。这种算法考虑了服务器的负载情况，优先将请求发送到负载较低的服务器上。

3. **最少响应时间算法（Least Response Time）：**

- 最少响应时间算法会将请求发送到响应时间最短的服务器上，以保证响应时间的最小化。这种算法通常需要负载均衡器记录每个服务器的响应时间，并动态调整请求的分配策略。

4. 哈希算法 (Hashing) :

- 哈希算法根据请求的某些属性 (如客户端IP地址、URL等) 计算哈希值, 并将请求发送到对应哈希值的服务器上。这种算法能够确保相同请求始终被发送到同一台服务器上, 适用于需要保持会话一致性的场景。

5. 加权轮询算法 (Weighted Round Robin) :

- 加权轮询算法在轮询算法的基础上引入了权重的概念, 不同的服务器具有不同的权重值。根据权重值的不同, 负载均衡器会调整请求的分配比例, 以实现负载均衡。

6. 拓展: hash环也是一种重要的负载均衡算法, 也可以提及。

服务治理和发现有没有做? 怎么做的?

回答要点: 一般是用第三方组件 (比如zookeeper) 来做, 当然raft-kv本身就可以用来做服务治理和服务发现, 所以rpc就没有单独做。

示例回答:

无

你这个RPC框架的序列化和反序列化中protobuf细节有没有了解

回答要点: 头部变长编码+自定义的压缩算法。

这里就可以牵扯到rpc中的编码方法, 目前是头部定长4字节, 可以优化成一个标志位+变长编码的方式: [参与issue链接](#)

示例回答:

大概了解了一下, 主要是其头部的变长编码+Google自己实现的一个高效的压缩算法。

测试

在集群数量变多的时候, Raft性能可能会下降, 这方面有没有思考过?

回答要点: 允许从follower读; rpc合并等raft落地的框架的优化。

示例回答:

无

有没有对性能进行过测试? 用的什么工具? 怎么测试的?

回答要点: perf火焰图。

示例回答:

这里回答一下火焰图的基本使用就差不多了, 如果大家没有使用过的话推荐大家去看篇博文入门了解基本操作和原理 (探针),

我这里给出一个初步的结果, 如果只有一个客户端: 并发几十, 大部分的损耗在rpc这边。多个客户端的结果没有测试。

分布式的内容相当多, 一方面需要不断补充学习, 另一方面需要择重点学习。

影响比较深的一个问题: Raft处理单向网络故障的时候的问题, 像这些比较偏的问题建议掌握主要问题之后有时间再看。

